



# TRIAGECORE

AI QA Engineer & CI Reliability Agent

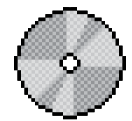


Shazil Rehman (23I-0095)  
Abdul Mohaimin (23I-0652)  
Taimoor Ahmed (23I-0639)

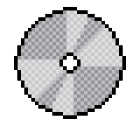
Supervisor: Dr. Uzma Mahar



# PROBLEM AND USERS

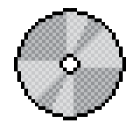


**Problem:** Blind AI self-healing can hide real regressions; TriageCore decides when a broken test or CI failure can be fixed automatically, and when it must escalate to a human.



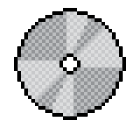
**Target Audience:**

- QA Engineers
- DevOps Engineers, QA Teams, and CI/CD Maintainers
- OpenSource Contributor
- Opensource Maintainers



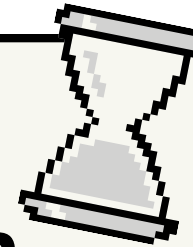
**Current Workflow and why it matters:**

- Engineering time lost to repetitive manual maintenance
- Existing “AI self-healing” tools are unreliable, and can mask real regression
- Developers at Google spend on average 3.7 hours to track down a single flaky test



**UN SDG-9 (Industry, Innovation, and Infrastructure):**

Eliminates redundant cloud compute waste from repeated flaky CI runs.



## Motivation

A 2025 research report titled The GenAI Divide from MIT's Project NANDA found that 95% of enterprise artificial intelligence (AI) projects fail to deliver measurable business value because they lack proper workflow integration and use static models that cannot learn



# STATE-OF-THE-ART & COMPARATIVE GAP ANALYSIS



Healinium

Proxys web browser and relocates UI elements when they change which is good but never checks the console errors or the app state after the click



Testim

Use ML to match locators. Gives a confidence score but that is all about how sure it is it has found the right button not after clicking it



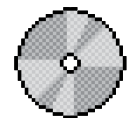
Trunk

Auto quarantines flaky tasks but lacks auto fix or bug classification features

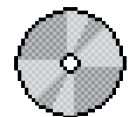
“Glaring lack of post action features”

| System                                          | Post-Action Safety Check | Confidence Scoring | QA + CI Unified Engine |
|-------------------------------------------------|--------------------------|--------------------|------------------------|
| Healenium (real product, open source)           | x                        | x                  | x                      |
| Prior FYP — Agentic API Test Generator          | x                        | x                  | x                      |
| Prior FYP — NLP-based UI Test Generator         | x                        | x                  | x                      |
| Prior FYP — AI Test Platform w/ Failure Tracing | x                        | x                  | x                      |
| TriageCore                                      | ✓                        | ✓                  | ✓                      |

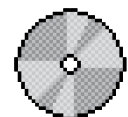
# RESEARCH TAKEAWAYS



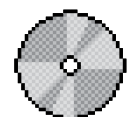
Biswas, S. (2026). Enhancing end-to-end test stability through AI-assisted self-healing: A case study of Playwright healer agent implementation.



FlaKat (2024): A machine learning-based categorization framework for flaky tests



Joseph, R. N. (2026).



Gruber, M., & Fraser, G. (2023)



# COMPLEX COMPUTING PROBLEM

|                                                                                                                                                                                                       |                                                                                                                                                                                                                                |                                                                                                                                                                                                           |                                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>Conflicting Requirements</div> <div>decision-latency and automation speed vs. the cost of a wrong automatic decision — a masked regression shipped, or a correct pipeline wrongly blocked.</div> | <div>No Obvious Solution</div> <div>a stale selector and a real regression produce the same surface evidence (a failed click, a red test) — no standard algorithm or lookup rule separates them without deeper analysis.</div> | <div>Ill-defined root cause</div> <div>tracing a CI failure to the one responsible commit across noisy, overlapping, multi-commit PR logs has no fixed procedure — it's an open diagnostic problem.</div> | <div>Significant Consequences</div> <div>a wrong auto-decision can silently ship a real bug behind a "passing" test, or misroute a correct fix — which is why every decision stays human-gated and logged for audit.</div> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

# PROPOSED SOLUTION AND CONTRIBUTION

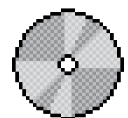
“Never silently resolve Ambiguity”

## Contribution



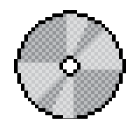
One shared Confidence Engine that decides, from evidence, whether to act automatically or escalate — used identically by a QA self-healing agent and a CI triage agent.

## Main Module



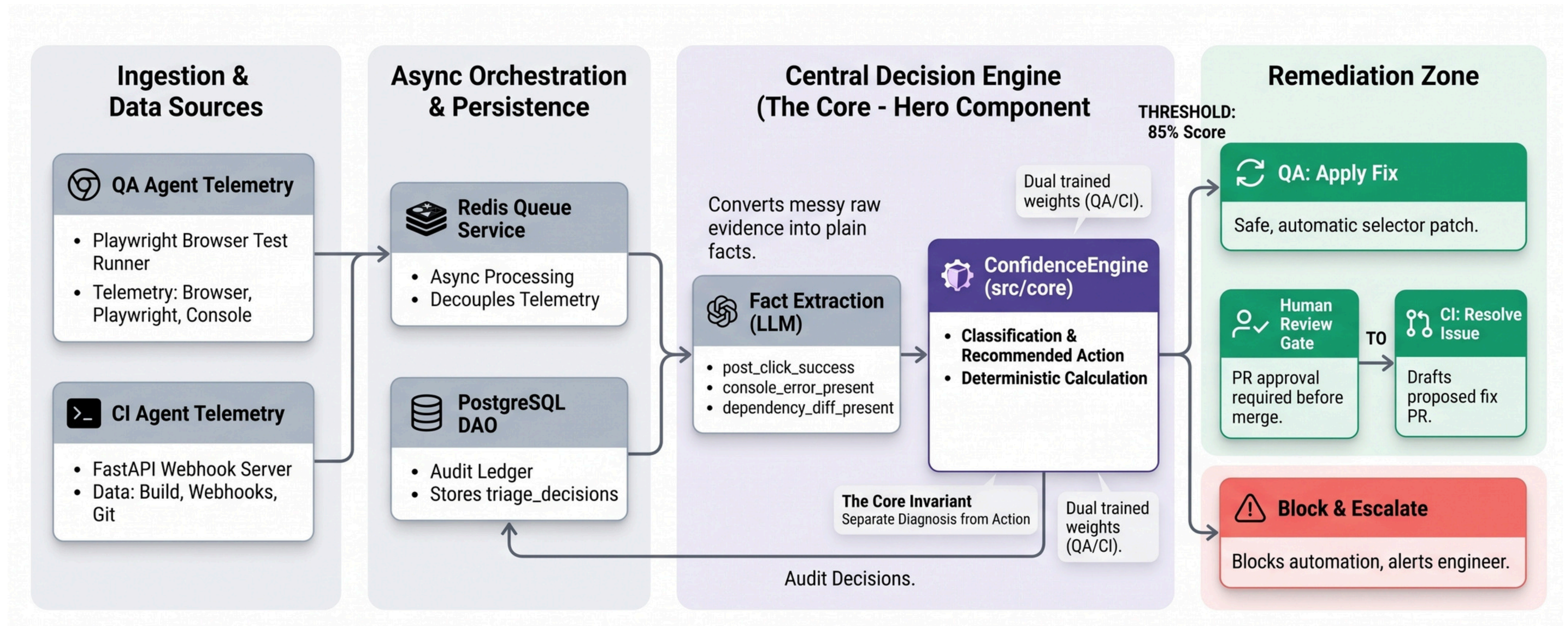
- Core Engine: confidence scoring & decision logic
- QA Agent: Playwright execution + selector relocation
- CI Agent: GitHub Actions log ingestion + commit tracing
- Persistence: Postgres audit log, Redis job queue

## How the Engine Decides



Extracts signals -> Computes weighted score from weights.yaml (0–100) → Outputs diagnostic-only classification -> Evaluates against threshold to choose recommended action (heal, escalate, auto-fix, flag).

# SYSTEM ARCHITECTURE



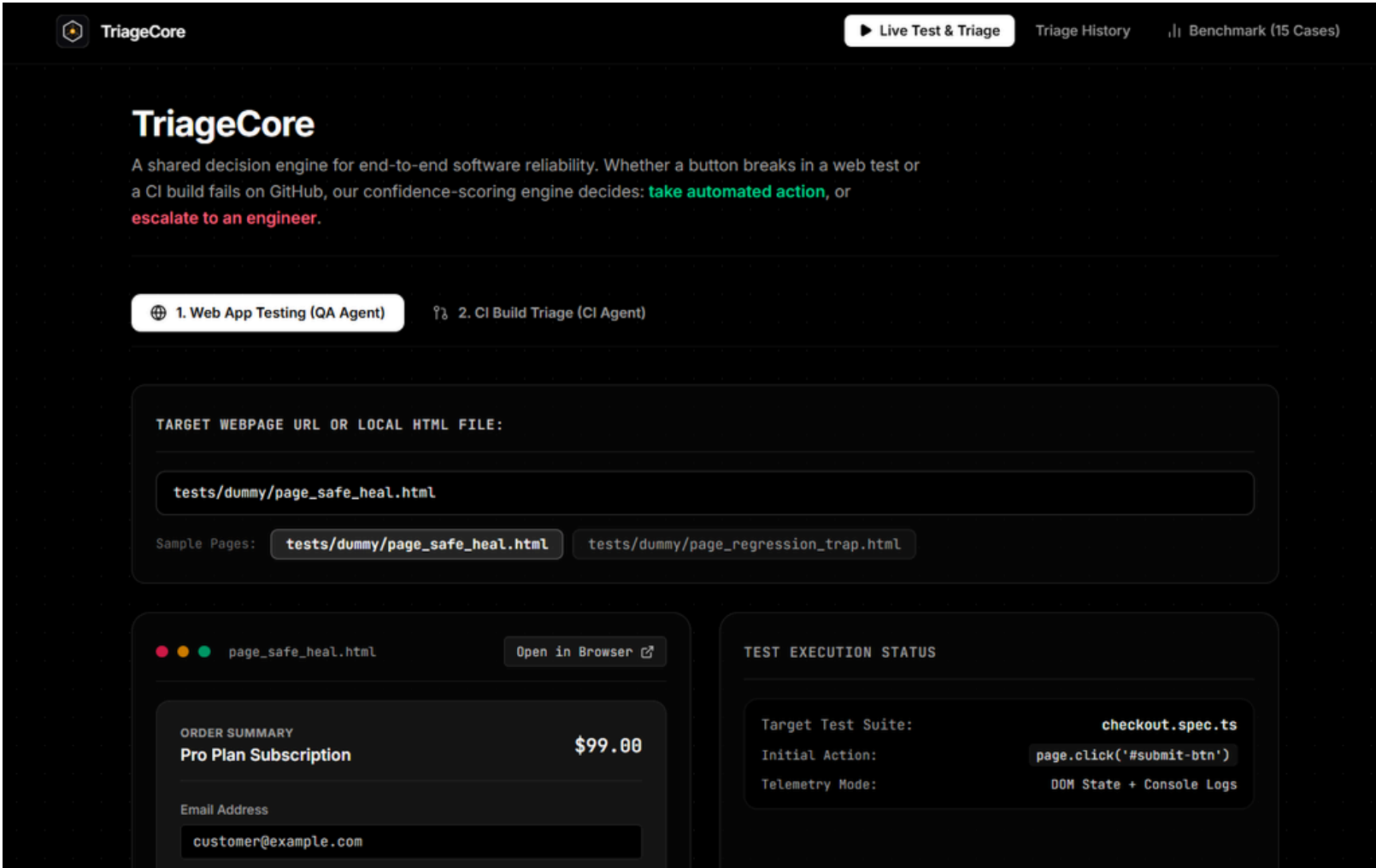
# POC-LITE / FEASIBILITY

## Feasibility

- A standalone script already runs a real browser (Playwright) and a real LLM call end-to-end on a local test page, completing in under 10 seconds
- This tests the riskiest assumption: can the system tell a safe fix apart from one that hides a broken click handler

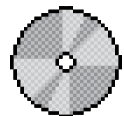


Link: [POC-lite Frontend](#)



# EVALUATION PLAN AND RISKS

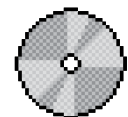
## Success Measure



- 50-Case Ground-Truth Benchmark Suite (15 flaky, 15 regressions, 10 stale selectors, 10 dependency breaks).
- Empirical Threshold Calibration (ROC Analysis) targeting < 5% False-Positive Rate.
- Non-Functional Performance Targets: Engine decision latency < 200ms; QA heal cycle < 10s; CI triage < 30s.
- 4-Week Live Trial (FYP-2): Deployed on 2–3 active open-source GitHub repositories to measure developer acceptance.

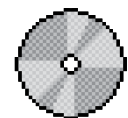


## Key risks and fallbacks:



- Selector matching fails → fall back to exact CSS selector, no AI healing
- CI log too large → fall back to extracting only the error stream
- Commit tracing ambiguous → flag the entire pull request instead of guessing
- LLM rate limits → fall back to a secondary provider

## Tool / API / GenAI boundary:

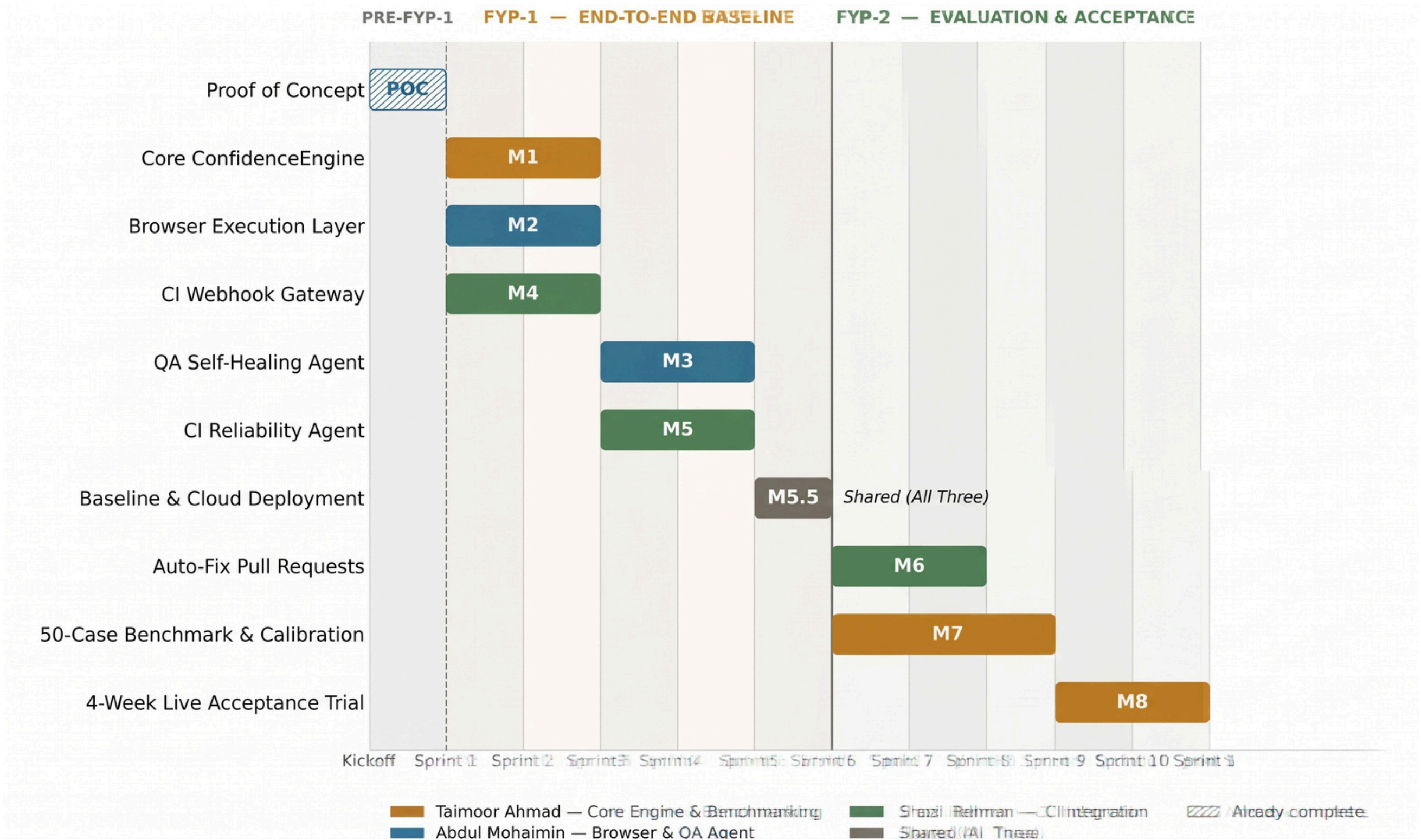


- LLM only parses unstructured text (HTML, logs); it never makes the heal/escalate decision
- GitHub API is read + open-PR only — the system never auto-merges



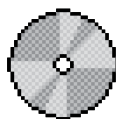
# WORK DIVISION AND ITERATION

## TENTATIVE PLAN



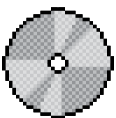
# ETHICS, LEGAL COMPLIANCE & AI GOVERNANCE

## Diversity, Equity and Inclusion



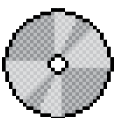
- Lowers onboarding friction for junior contributors and reduces repetitive developer burnout.

## Human-in-the-Loop AI Governance

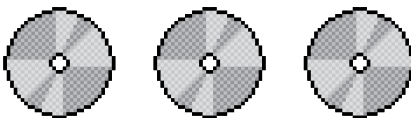


- The system never auto-merges a pull request
- Every decision is logged with a reasoning trace, so a human can audit why a choice was made

## Output Stage



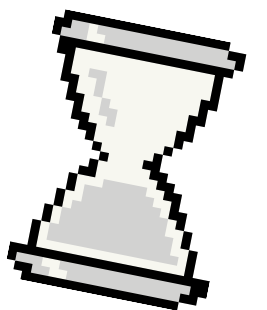
- All GitHub API use complies with GitHub's Terms of Service
- Webhook payloads are verified (HMAC signature) before processing
- No personal data is retained — only the triage decision and its reasoning trace



# REFERENCES

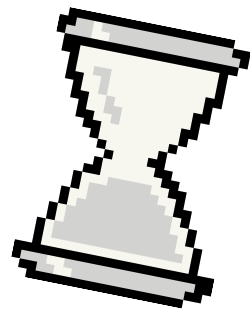


- Biswas, S. (2026). Enhancing end-to-end test stability through AI-assisted self-healing: A case study of Playwright healer agent implementation. International Journal of Scientific Engineering and Research, 14, 1-12. [https://www.researchgate.net/publication/399515915\\_Enhancing\\_End-to-End\\_Test\\_Stability\\_Through\\_AI-Assisted\\_Self-Healing\\_A\\_Case\\_Study\\_of\\_Playwright\\_Healer\\_Agent\\_Implementation](https://www.researchgate.net/publication/399515915_Enhancing_End-to-End_Test_Stability_Through_AI-Assisted_Self-Healing_A_Case_Study_of_Playwright_Healer_Agent_Implementation)
- Lin, S., Liu, R. Z. H., & Tahvildari, L. (2024). FlaKat: A machine learning-based categorization framework for flaky tests. arXiv preprint. <https://arxiv.org/abs/2403.01003>
- Gruber, M., & Fraser, G. (2023). Debugging flaky tests using spectrum-based fault localization. arXiv preprint. <https://arxiv.org/abs/2305.04735>
- Joseph, R. N. (2026). Beyond LLM-based test automation: A zero-cost self-healing approach using DOM accessibility tree extraction. arXiv preprint. <https://arxiv.org/abs/2603.20358>





**THANK YOU**  
**ANY QUESTIONS?**



Link: [POC-lite Frontend](#)